

Aptly: Making Mobile Apps from Natural Language

Evan W Patton*

Computer Science and Artificial
Intelligence Lab
Massachusetts Institute of Technology
Cambridge, USA
ewpatton@mit.edu

Robin Liu

Massachusetts Institute of Technology
Cambridge, USA
robinl21@mit.edu

David Y Kim*

Computer Science and Artificial
Intelligence Laboratory
Massachusetts Institute of Technology
Cambridge, USA
dyjkim@mit.edu

Arianna Scott

MIT App Inventor
Massachusetts Institute of Technology
Cambridge, USA
acscott@mit.edu

Ashley M Granquist*

Computer Science and Artificial
Intelligence Laboratory
Massachusetts Institute of Technology
Cambridge, USA
ashleymg@mit.edu

Jennet Zamanova

Massachusetts Institute Technology
Cambridge, USA
zamanova@mit.edu

Harold Abelson

Massachusetts Institute Technology
Cambridge, USA
hal@mit.edu

Abstract

This paper introduces Aptly, a platform designed to democratize mobile app development, particularly for young learners. Aptly integrates a Large Language Model (LLM) with App Inventor, enabling users to create apps using their natural language. User's description is translated into a programming language that corresponds with App Inventor's visual blocks. A preliminary study with high school students demonstrated the usability and potential of the platform. Prior programming experience influenced how users interact with Aptly. Participants identified areas for improvement and expressed a shift in perspective regarding programming accessibility and AI's role in creative endeavors.

CCS Concepts

• **Human-centered computing** → **User interface programming**.

Keywords

Computational Action, Large Language Model, Block Programming, Mobile Application

ACM Reference Format:

Evan W Patton, David Y Kim, Ashley M Granquist, Robin Liu, Arianna Scott, Jennet Zamanova, and Harold Abelson. 2025. Aptly: Making Mobile Apps from Natural Language. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3706599.3720081>

* Authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3720081>

Yokohama, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3706599.3720081>

1 Introduction

Our core goal is to empower young people to develop technology worth making novel, digital solutions to problems young people face every day in their lives, their communities, and their world. Mobile applications serve as a powerful medium for such engagement given the widespread adoption of smartphones and tablets across diverse demographics, including under-represented communities [4]. However, the traditional app development process, which often requires a solid foundation in computer science, has historically excluded many aspiring creators from participating. Initiatives like App Inventor have democratized mobile app creation by introducing a user-friendly, visual programming environment. This block-based approach allows anyone, regardless of technical background, to develop applications by assembling geometric shapes representing code blocks [3, 12]. The tool not only fosters creativity among novice programmers but also encourages a more inclusive approach to technological education and development [1]. Tissenbaum et al. call the process by which people leverage technology like App Inventor to make societal impact *computational action* [10].

The advent of large language models (LLMs) [11] and their ability to generate code [8, 9] has opened new avenues for mobile app development. Capitalizing on this technological advancement, our research introduces Aptly, a platform that allows users to create mobile applications using natural language. For example, a user could simply state, “*Make me an app that translates English to Spanish,*” and Aptly would automatically generate a functional app based on this input. In a high level, the platform works as the following: The user describes their desired app or modification, the LLM translate that into a textual language that has a one-to-one corresponding with App Inventor's blocks. Finally, the App Inventor platform compiles the blocks into a functional mobile app. This paper details

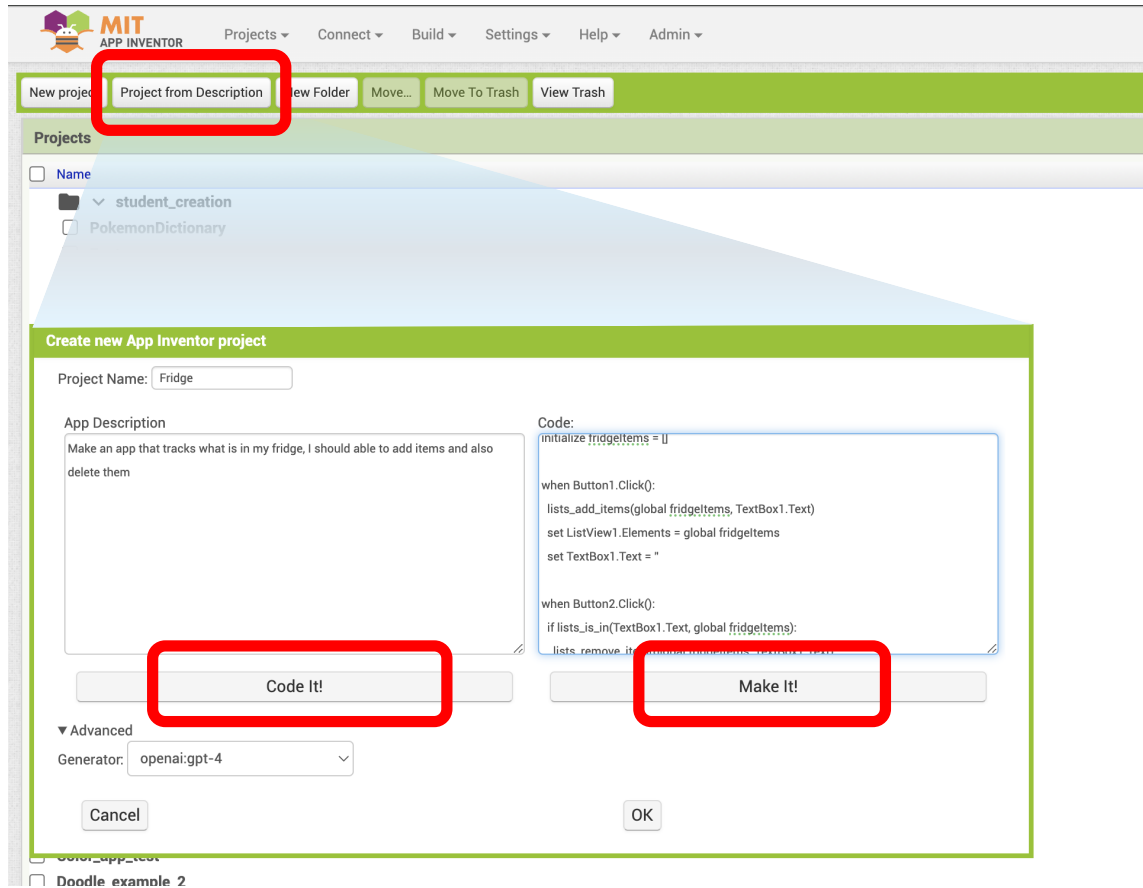


Figure 1: The main Aptly interface is integrated with MIT App Inventor. On the top left there is “Project from description” button. Here the user typed in “Make an app that tracks what is in my fridge, I should be able to add items and also delete them”.

the design of our platform, discussing the development of the Aptly Language and our method of integrating it with an LLM to produce the desired syntax. We conclude by presenting preliminary findings from its deployment among high school students.

2 User Interface

The Aptly user interface (Figure 1), integrated with App Inventor platform, introduces a new button labeled “Project from description”. When clicked, opens a window allowing the user to select a model for code generation and input a description of the app they wish to create. After detailing their app, users click “code it” to activate a large language model that generates the corresponding code. This process will be explored in greater detail later in the paper. Once the code is generated, it appears in a textbox on the right, where users can make manual adjustments as needed. After refining the code to their satisfaction, users press “make it” which automatically parses the code into a format compatible with App

Inventor. The user is then redirected to the standard App Inventor interface, where they can continue developing the app using the designer and blocks features.

Users can describe the layout and specify the functionality of each user interface (UI) component within their mobile application. For example, users might design an application such as “*Make an app with a text box and a button. When the button is clicked, the app says the words in the text box out loud. If the textbox is empty say “I have nothing to say”*”. Another instance is “*Create an app with a button in the center and a label below the button that tracks the number of times the button is clicked.*”. Additionally, Aptly supports the development of applications where the user does not need to specify detailed functionalities. For example, a user can simply request, “*make a calculator app*” and Aptly generates the application based on this high-level directive. This capability demonstrates Aptly’s ability to operate both as a detailed application development tool where specific UI behaviors are defined and as a more abstract tool that interprets and implements broad application concepts.

Users also have the capability to further refine their applications. The user can summon the pop-up window and specify the modifications they desire. For example, a user might request specific adjustments like *“Make the font of the label three times larger”* or more general changes such as *“Make the button bigger”* or even abstract alterations like *“Change the color of the button to the color of a melon”*. Aptly extends beyond mere automation by augmenting the creative process itself. As an intelligent agent powered by a large language model (LLM), Aptly assists users in achieving designs that surpass their initial conceptions. For example, the optimal color schemes for colorblind individuals might not be readily apparent to the average user. However, by using Aptly, a user can simply request that the UI color scheme be adjusted to be colorblind friendly, and the platform will automatically apply an appropriate palette based on its extensive knowledge base.

3 Platform Design

3.1 The Aptly Language

```
Screen1 = Screen()
HA1 = HorizontalArrangement(Screen1)
Label1 = Label(HA1, Text = "Weight in lbs: ")
EarthWeight = TextBox(HA1, NumbersOnly = True)
Label2 = Label(Screen1, Text = "Select Planet:")
PlanetList = ListView(Screen1, ElementsFromString = "Mercury,
    Venus, Mars, Jupiter, Saturn, Uranus, Neptune")
Calculate = Button(Screen1, Text = 'Calculate')
PlanetaryWeight = Label(Screen1)

initialize gravities = {"Mercury": 0.38, "Venus": 0.91, "Mars":
    0.38, "Jupiter": 2.34, "Saturn": 0.93, "Uranus": 0.92,
    "Neptune": 1.12}

to compute_weight(earth_lbs, planet):
    return earth_lbs * dictionaries_lookup(planet, global gravities,
        'not found')

when Calculate.Click():
    set PlanetaryWeight.Text = call compute_weight(EarthWeight.Text,
        PlanetList.Selection)
```

Listing 1: An app to calculate weights on different planets expressed in Aptly.

The Aptly language is designed to be used with LLMs. It is a textual representation of App Inventor programs inspired by Python. The decision to base Aptly on Python was made for two reasons:

- (1) Python’s pseudocode-like syntax closely aligns with the readability and structure of App Inventor blocks, facilitating intuitive understanding and use.
- (2) LLMs have been extensively trained on Python code, enhancing their ability to generate accurate and functional code in a similar syntax.

Given these attributes, we hypothesize that generative models can be effectively tuned to interface with Aptly, leveraging its Python-like structure. However, it is important to note that, while Aptly draws inspiration from Python, it is distinct. Aptly has been specifically constrained to ensure a one-to-one correspondence with App

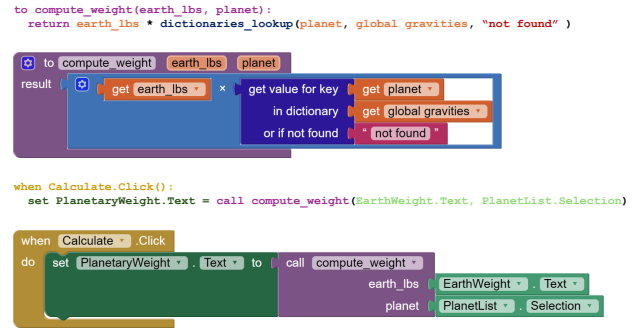


Figure 2: An example of the Aptly code in Listing 1 and its correspondence to the App Inventor interface.

Inventor: every valid App Inventor program is also a valid Aptly program and vice versa. See Listing 1 for an example Aptly program and Figure 2 to showcase how the code is represented as blocks.

3.2 Few-shot Prompt Engineering

LLMs operate based on “prompts” that are crucial inputs that significantly influence their performance. The effectiveness of these models hinges on the quality of the prompts, a practice known as prompt engineering [7]. A common method of prompt engineering involves “few-shot” prompts, where a model is given a limited number of solved task examples as part of its input. This approach is often interpreted as the model “learning” the task in real time from these few examples [2]. Aptly uses Open AI’s LLMs, which are capable of generating computer code from natural language descriptions in several programming languages.

When the user requests an app with their natural description, we synthesize a prompt which is a natural language description (denoted as D) of the desired app to be created, together with a set of example pairs, such as the following $\langle\langle d_1, c_1 \rangle\rangle \langle\langle d_2, c_2 \rangle\rangle \dots \langle\langle d_k, c_k \rangle\rangle$, where d_i is the description of application i , and c_i is the corresponding Aptly code of the description for application i . The example pairs come from a database of unique Aptly examples compiled by the team from apps created on the App Inventor platform. The example pairs are not expected to include the literal description D to be processed nor the actual Aptly Code to be generated which would make the generation task trivial. Instead, generative models use the provided examples to guide their processing in generating new original output.

Aptly’s method for providing relevant prompts to accompany a description D is to provide a set of example application descriptions paired with appropriate Aptly Code. We use semantic similarity to highlight the “appropriateness” of code and description. The method relies on having a way to automatically measure the similarity of text descriptions and the program. For computing the similarity of the text description and the program, we use embeddings of data elements, which are numerical representations of concepts converted to number sequences [6]. In our scenario, an embedding represents the semantic meaning of a natural language description or code. To measure the similarity of two items, one

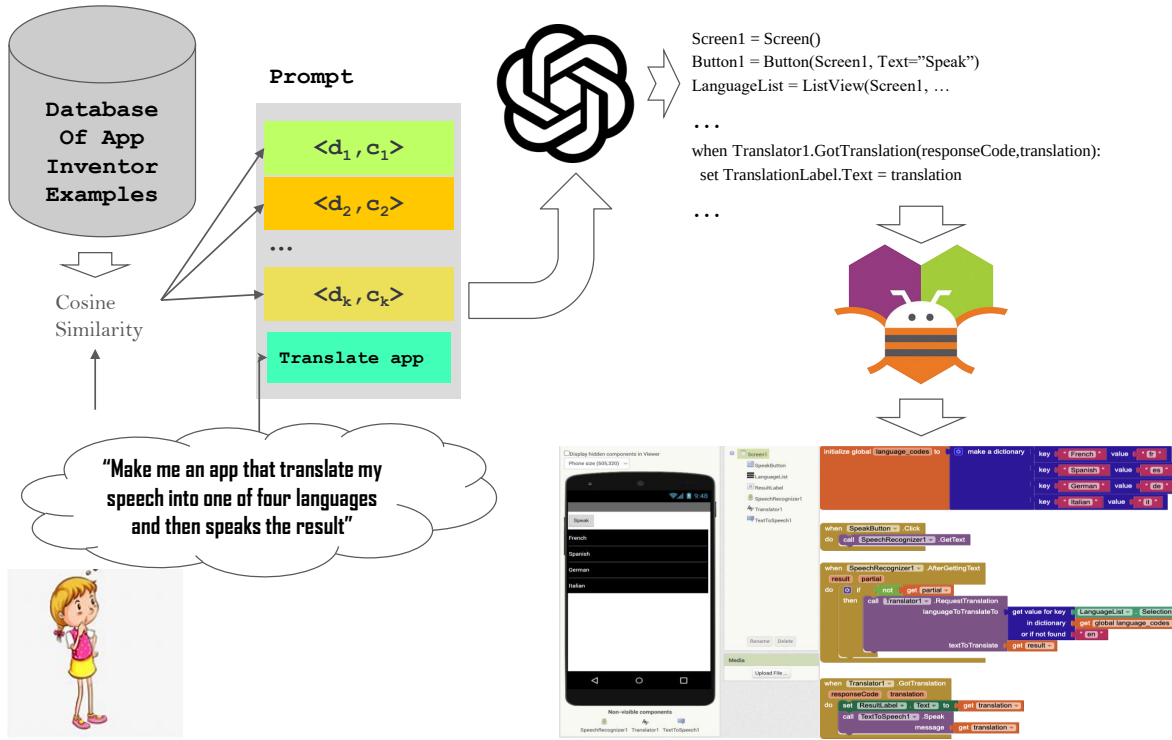


Figure 3: When the user requests an application with its textual description, we automatically synthesize the prompt by adding several example pairs along with the desired application’s textual description. This constructed prompt is fed into one of OpenAI’s GPT-X models as an input, which outputs code that can be converted into a fully functional mobile application

embeds them in the same vector space and takes the distance between the two embedding vectors as a measure of similarity (small distance implies high similarity, while large distance implies low similarity). We used the cosine distance, which reflects the angle between the vectors, to compute the similarity between two vectors.

We employ OpenAI’s Text/Code Embedding model to generate embeddings from descriptions and associated code. These neural network models utilize Contrastive pre-training, a training method that functions by clustering in a vector space [5]. Specifically, it draws predefined positive examples (i.e., matching text and code) closer together, while pushing negative examples (i.e., contrasting text and code) apart. This technique enhances our model’s ability to identify and select the example pair most relevant to a user’s specified description. We feed the synthesized prompt into GPT, which outputs the Aptly Code corresponding to the user’s requested app description. We can then convert the generated code into App Inventor blocks to generate a fully functional application (Figure 3).

4 Pilot User Study

To evaluate the effectiveness and user experience of our tool, we conducted a study involving 10 high school students with the following demographics:

- **Age:** 17 (5), 18 (5)
- **Gender:** Female (8), Male (2)

- **General programming experience:** None (3), < 1 year (2), 3-4 years (3), 5+ years (2)
- **Familiarity with block-based programming:** None (3), Beginner (2), Familiar (5)

This group was selected to yield insights into how students with different expertise levels and app creation familiarity engage with our tool. The study duration ranged from 40 to 60 minutes per participant. Initially, we introduced the students to Aptly, demonstrating how to create a basic app with interactive features such as a button and a label, and how to modify properties like color and font using the tool’s editing capabilities. We then assigned two distinct tasks to assess the tool’s utility and the participants’ ability to leverage it.

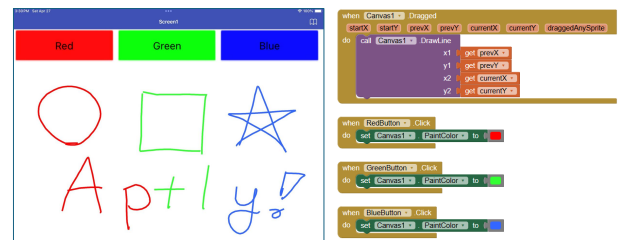


Figure 4: App Inventor Basic Tutorial: Digital Doodle

4.1 Recreation Task

Students were shown “Digital Doodle” (Figure 4), a beginner-level app from App Inventor, and asked to recreate it using Aptly. This task was designed to observe whether participants could effectively translate a clear visual concept into a functional app through verbal descriptions, reflecting their understanding and communication of app design concepts. The task required the platform’s flexibility in understanding and executing a range of instructions, from detailed descriptions specifying colors and functionalities of buttons (e.g., *An app with 3 buttons (one red, one blue, one green), and a pen drawing feature with a blank “drawing” space beneath it. When the mouse is dragged in the blank space, create a line where the drag is. When red is clicked, the pen color is red. When blue is clicked, the pen color is blue. When green is clicked, the pen color is green.*) to succinct commands like “Draws with RGB.”

Participants employed different strategies in app creation, some opting for a more incremental approach using Aptly’s editing features. For instance, one participant began with a basic layout and progressively added functionalities and aesthetic adjustments, such as alignment and color changes, through subsequent commands. This iterative process was particularly favored by participants with programming backgrounds, reflecting a workflow similar to traditional software development where features are built and refined over time. The study also uncovered challenges related to the precision of language used in commands. Some participants struggled to articulate their desired app functionalities clearly, leading to discrepancies between their intentions and the app’s behavior. For example, a participant’s command to “draw a line when I click a button” was too vague without specifying the color interaction.

4.2 Scenario-Based Task

Participants were given a scenario. Specifically, they were asked the following scenario: **“In your class you have a friend that is visually impaired and is having trouble with their math class. You want to help that student by creating a calculator app that will help them with their addition and subtraction. Try using Aptly to create that app. Feel free to use the editing and also manually drag and drop the blocks.”** This task aimed to evaluate participants’ ability to ideate and articulate solutions to real-world problems using Aptly. It also encouraged them to consider accessibility features, such as voice commands and easy-to-navigate interfaces, enhancing their problem-solving and design-thinking skills.

Participants provided initial descriptions that ranged from simple one-liners like “Make an app that is a calculator” then incrementally build and refine their app’s user interface and functionalities using Aptly’s editing tools to detailed paragraphs specifying the layout and functionality, including voice recognition and auditory feedback (e.g., *“Have a third button that hears one number, then hears either add or subtract, then hears a second number, after all three components are heard, then display the sum or difference of the two numbers in the top, then display the number in the display box too”*).

An interesting observation is that the participants’ creative solutions for accessibility features were varied. While most of the participants added audio systems for the accessibility feature, one

innovative approach involved using screen taps and phone vibrations to communicate numbers and results (*“Make each displayed element bigger. Also, when the result is displayed, vibrate the number of times of the result”*).

5 Discussion

Observations revealed valuable insights into user behavior and platform design. Some participants exhibited impatience with the system response times as it generally takes around 3 ~ 10 seconds for a change to reflect. They often proceed to prompt subsequent changes if an immediate implementation of the edits was not observed. Challenges arose when Aptly misunderstood user commands; while some participants attempted to rephrase their inputs multiple times using Aptly editing, others opted to manually make the changes. Notably, more experienced programmers frequently preferred manual adjustments and issued a higher number of commands compared to their less experienced counterparts. These seasoned programmers expressed a desire for greater control, including access to error messages to better understand the system’s handling of their commands. This feedback highlights a tension between designing intuitive interfaces suitable for beginners and providing detailed control and feedback that experienced users seek. This dichotomy suggests that while simplifying interfaces can benefit novices, it may frustrate advanced users who expect deeper interaction and transparency from the development tools.

5.1 Participant responses

To capture the changes in participants’ perceptions before and after using Aptly, we asked the participants two questions before and after the experiment.

- **“Programming is accessible for everyone”**: Prior to the workshop, students’ views on programming accessibility varied, with some expressing skepticism about universal accessibility due to factors such as technological availability and socioeconomic barriers. Common responses ranged from “somewhat disagree” to acknowledging partial agreement. After using Aptly, participants generally showed a nuanced view, with some noting improvements in their perceptions of accessibility. Notably, though some recognized improvements, they still highlighted limitations, particularly for individuals who might lack basic digital literacy or access to necessary hardware.
- **“Artificial Intelligence can be a powerful tool to help people in programming”**: Initially, responses indicated a positive sentiment towards AI as a powerful aid in programming, with a slight variety hinting at potential job displacement concerns. After the experiment, reflections were more detailed, emphasizing the practical benefits and challenges observed during the use of AI-driven tools like Aptly. Participants appreciated AI’s potential to simplify the programming process, especially for beginners, yet also discussed the importance of understanding underlying logic beyond AI assistance.

5.2 Limitation of Aptly based on responses

In our study, participants identified several limitations of the Aptly platform, providing valuable insights into its operational shortcomings and areas ripe for development. Notably, Aptly struggled with complex command executions such as playing specific audio files, drawing predefined shapes, and making API calls. Improvements in the user interface were recommended to make the tool more intuitive. Especially users were confused whether their prompt was not clear enough or the system is not capable to conduct their request. Enhanced error messaging and user feedback, particularly for users with programming experience, were noted as critical for allowing more effective debugging and application refinement. These findings indicate a significant need for advancements in natural language understanding within Aptly to accommodate a broader array of user instructions and technical demands.

6 Conclusion

This research introduces Aptly, a platform designed to empower individuals, particularly young learners, to create mobile applications that positively impact their lives and communities. By integrating a Large Language Model (LLM) with the App Inventor platform, Aptly simplifies the app development process. This involved engineering a novel textual language that directly corresponds with App Inventor's block-based programming and employing effective prompt engineering to teach the model the platform's functionality.

A preliminary study with ten high school students revealed valuable insights into Aptly's usability and potential. Notably, programming experience influenced how students interacted with the platform. Participants identified areas for improvement, such as enhanced error handling and expanded capabilities like audio file integration. Significantly, Aptly shifted many participants' perspectives on both programming accessibility and the role of AI in this process. This study provides crucial guidance for Aptly's continued development and lays the groundwork for future research exploring its potential to facilitate computational action and empower a wider range of individuals to engage in app creation.

References

- [1] Rosa Bottino, Augusto Chioccariello, and Laura Freina. 2020. *Computational Thinking in Primary School Through Block-Based Programming*. Springer Singapore, Singapore, 147–166. doi:10.1007/978-981-15-4952-6_10
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.
- [3] Siti Nor Hafizah Mohamad, Ahmed Patel, Rodziah Latih, Qais Qassim, Liu Na, and Yiqi Tew. 2011. Block-based programming approach: challenges and benefits. In *Proceedings of the 2011 International Conference on Electrical Engineering and Informatics*. 1–5. doi:10.1109/ICEEL.2011.6021507
- [4] Philip M. Napoli and Jonathan A. Obar. 2014. The Emerging Mobile Internet Underclass: A Critique of Mobile Internet Access. *The Information Society* 30, 5 (oct 2014), 323–334. doi:10.1080/01972243.2014.944726
- [5] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005* (2022).
- [6] Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). Association for Computational Linguistics, Doha, Qatar, 1532–1543. doi:10.3115/v1/D14-1162
- [7] Laria Reynolds and Kyle McDonell. 2021. Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems (CHI EA '21)*. Association for Computing Machinery, New York, NY, USA, Article 314, 7 pages. doi:10.1145/3411763.3451760
- [8] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1 (Lugano and Virtual Event, Switzerland) (ICER '22)*. Association for Computing Machinery, New York, NY, USA, 27–43. doi:10.1145/3501385.3543957
- [9] Haoran Su, Jun Ai, Dan Yu, and Hong Zhang. 2023. An Evaluation Method for Large Language Models' Code Generation Capability. In *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*. 831–838. doi:10.1109/DSA59317.2023.00118
- [10] Mike Tissenbaum, Josh Sheldon, and Hal Abelson. 2019. From computational thinking to computational action. *Commun. ACM* 62, 3 (2019), 34–36.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (Long Beach, California, USA) (NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [12] David Wolber, Harold Abelson, and Mark Friedman. 2015. Democratizing Computing with App Inventor. *GetMobile: Mobile Comp. and Comm.* 18, 4 (jan 2015), 53–58. doi:10.1145/2721914.2721935